D.3.1 Midterm Project Status Report

PlanER: Cost-Based Optimization of Hybrid Classical–LLM Entity Resolution Plans in a Document Database

Course: COMP 8157 Advanced Database Topics, Fall 2026 · **Team:** 07 · **Authors:** Shashang Patel (patel3cd@uwindsor.ca), Nauman Syed (syed9i@uwindsor.ca) · School of Computer Science, University of Windsor · **Date:** November 3, 2026 · **Version:** 1.0

1. Project Progress Breakdown

PlanER treats entity resolution (ER) as a declarative request that a cost-based optimizer compiles into a physical plan over MongoDB: a blocking access path, an ordered cascade of matchers, escalation thresholds and a clustering method. At the midpoint of the term the project is on schedule. Phase 1 (environment, data and the four blocking access paths) and Phase 2 (classical matchers, the local LLM tier and the verdict cache) are complete, and Phase 3 (the optimizer) began on November 2 as planned in the D.2.1 proposal. A working prototype already runs a request end to end under a fixed, hand-chosen plan: it loads two collections, generates candidate pairs through any of the four indexes, decides them with the matcher cascade, and groups matches into entities. What remains is to let the optimizer choose that plan instead of the developer, which is the project's central contribution.

1.1 Task Breakdown Structure (TBS)

The table lists the twenty tasks in execution order, grouped by phase. Each task is marked Completed, In Progress or Remaining, and the Depends on column gives the tasks that must finish first. Requirement identifiers refer to the D.4.1 User Requirements document.

ID	Task	Status	Depends on	Remarks / evidence
T1	Docker environment: MongoDB 8.2, mongot, Ollama, Python 3.11 app	Completed	–	One Compose file gives both members an identical stack (NFR-10)
T2	Ingest and normalize benchmarks; tokens, blocking key, 384-d embeddings	Completed	T1	DBLP–ACM, Abt–Buy, Amazon–Google, DBLP–Scholar loaded with mongoimport (FR-2)
T3	B-tree blocking on key + $lookup	Completed	T2	FR-3; explain() confirms index use
T4	Multikey token-index blocking (shared rare tokens)	Completed	T2	FR-4
T5	Full-text blocking with $search (BM25 top-k)	Completed	T2	FR-5; runs on mongot
T6	Vector blocking with $vectorSearch (HNSW top-k)	Completed	T2	FR-6; Atlas fallback kept behind same interface
T7	Blocking profiling: pair completeness, reduction ratio, latency, index size	Completed	T3–T6	Results and explain() written to run_log (FR-16)
T8	String-similarity rule matcher (RapidFuzz)	Completed	T2	FR-7; also produces the similarity features for T9–T10
T9	Fellegi–Sunter matcher fitted by EM (own implementation)	Completed	T8	FR-8; no labels needed
T10	Random forest on similarity features	Completed	T8	FR-9; trained on a split disjoint from test (NFR-7)
T11	Local LLM tier (Llama 3.1 8B, Qwen 2.5 7B) with llm_cache	Completed	T1	FR-10, FR-12; greedy decoding, fixed prompt
T12	Fixed-plan prototype: blocking → cascade → connected components; Streamlit page	Completed	T7, T11	End-to-end prototype required for this deliverable
T13	Per-matcher profiling of cost, escalation rate, precision and recall	In Progress	T8–T11	Feeds the optimizer's cost model (FR-13)
T14	Stratified sampling and plan cost/F1 estimation (Eq. 1)	In Progress	T7, T13	Phase 3, due Nov 15
T15	Plan enumeration with Pareto pruning and plan selection	Remaining	T14	FR-14, FR-15

T16	Optimizer-driven execution; predicted vs observed cost and F1	**Remaining**	T12, T15	FR-19, FR-20
T17	Transitivity-violation detection and repair	**Remaining**	T12	FR-18; Phase 4
T18	Hosted-LLM tier and WDC Products (stretch goals)	**Remaining**	T11, T2	FR-11, FR-23
T19	Full evaluation vs LLM-only, best classical and LOTUS-style baselines	**Remaining**	T16, T17	FR-22; Phase 4, due Nov 29
T20	Optimizer controls in demo; D.4.3–D.4.5 and final report	**Remaining**	T16, T19	Phase 5, due Dec 1

1.2 Evaluation Focus: Sequencing and Progress

The sequence follows the data dependencies of the system rather than the order of the requirements. Nothing can be blocked before records exist with normalized fields and embeddings, so the environment and ingestion (T1, T2) came first. The four access paths (T3–T6) depend only on those derived fields and were built in parallel, then profiled together (T7) so that they could be compared on equal terms. The matchers come next because they consume candidate pairs, and the rule matcher (T8) was built before Fellegi–Sunter and the random forest because it computes the similarity features both of them use. The LLM tier (T11) depends only on the environment, which let it be developed at the same time as the classical matchers. The optimizer (T13–T15) must come after all of these, because it estimates plans from measurements of the very operators it schedules. Evaluation (T19) comes last because it needs the optimizer's plans to compare against the baselines.

Since the proposal was approved on October 4, twelve of twenty tasks have been completed and two more are underway, covering the whole storage, indexing and matching layer of the system. The prototype (T12) is the strongest evidence of progress: it already turns a request into entity clusters inside MongoDB, so the remaining work replaces a human decision (which plan to run) with an optimizer decision rather than building missing parts of the pipeline.

Novelty and insights so far. Building the access paths changed how we model the plan space. The two top-k paths ($search and $vectorSearch) have a parameter k that directly trades pair completeness for candidate count, so k has become part of the physical plan rather than a fixed setting, much as a database optimizer chooses a scan range. The B-tree path, in contrast, has no such knob: its recall is fixed by how well the blocking key fits the dataset. This confirms the proposal's premise that blocking is an optimizer decision and not a preprocessing step, because no single path or k suits every dataset. A second insight concerns cost accounting. Because the verdict cache is keyed by a hash of the model, prompt version and pair, a pair answered once is free in every later plan, so the cost model must count only pairs that miss the cache. This is a direct analogue of a materialized view, and it changes which plans are cheapest when several plans are explored over the same data.

1.3 Schedule Status and Risk Handling

Figure 1 places the current position against the five-phase schedule from the proposal. Both completed phases finished within their two-week windows, and Phase 3 started on its planned date, so the project is on schedule with no slippage. The phase boundaries were chosen so that each phase ends with a measurable artifact, which is what allows this report to tie every completed task to a demonstrable result.

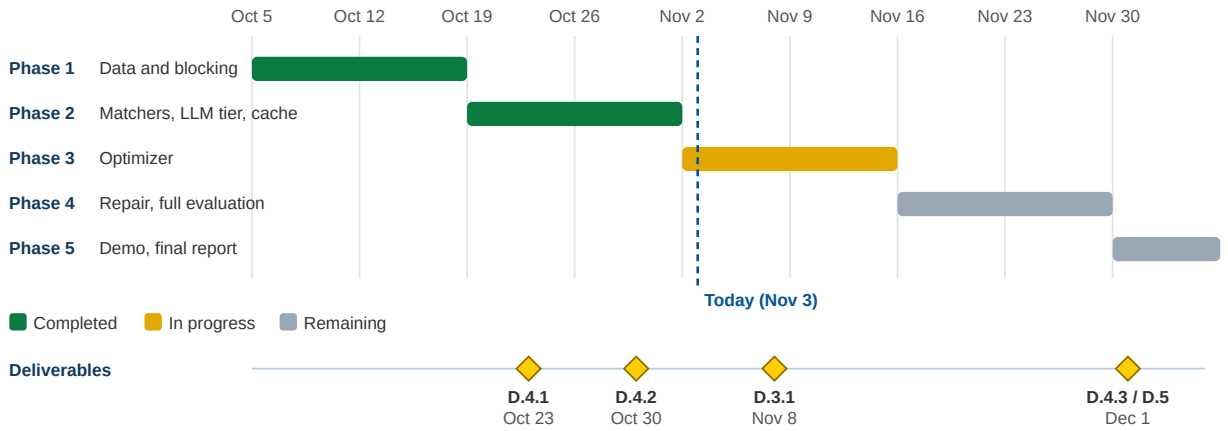


Figure 1. Project schedule by phase (D.2.1 timeline) with current status and deliverable due dates.

Two risks identified in the proposal shaped the work of the first two phases. The first is that full-text and vector search in MongoDB Community Edition are in public preview and run on the separate mongot process. The team mitigated this by placing all four access paths behind one interface, `candidates(request, k)`, so that the vector path can move to the free Atlas tier without changing any other code. The second is that 7–8B local models are slow on laptop hardware. This is handled structurally rather than by buying faster hardware: the LLM tier only ever receives pairs that every cheaper matcher has escalated, and every verdict is stored in the cache, so a pair is never sent to a model twice. The main open risk for the second half of the term is the accuracy of the optimizer's estimates when matches are rare in the sample. Stratified sampling by similarity score addresses this, and the team will vary the sample size in Phase 3 to measure how estimation error shrinks, reporting poor calibration as a finding if it occurs.

2. GitHub Repository and Version Control Evidence

All development is tracked in a shared GitHub repository: **[GitHub repository URL]**. The default branch holds only reviewed code. Each task in the TBS is developed on its own feature branch named after the task, for example `feature/T6-vector-blocking`, and merged through a pull request that the other member reviews, so every merged component has a recorded author and reviewer. Commit messages describe the change and cite the task identifier, for example "T9: fit Fellegi–Sunter m/u probabilities with EM", which ties the commit history to the breakdown in Section 1 and lets an evaluator trace each completed task to its commits.

The repository is organized by subsystem so that its layout mirrors the logical architecture in the D.4.2 design document. The table below gives the top-level structure.

Folder	Contents
docker/	Compose file and configuration for mongod, mongot, Ollama and the app container
data/	Download and mongoimport scripts for each benchmark (raw data is not committed)
src/planer/	Python package with one module per subsystem: ingest, blocking, matchers, optimizer, clustering
ui/	Streamlit demo application
experiments/	Scripts that run blocking and matcher profiling and write to run_log
tests/	Unit tests for each blocking path and matcher interface
docs/	Deliverables D.1 to D.4.2 and this report

Member contributions. As set out in the proposal, one member leads storage and blocking and the other leads matching and optimization, and both review each other's work. Shashang Patel led the storage and blocking layer: he built the Docker environment (T1), the ingestion and normalization pipeline including embeddings (T2), the four blocking access paths (T3–T6) and their profiling with explain() output (T7), and assembled the fixed-plan prototype and Streamlit page (T12). Nauman Syed led the matching layer: he implemented the rule matcher (T8), the Fellegi–Sunter model with expectation–maximization (T9), the random forest (T10) and the local LLM tier with the verdict cache (T11), and he is

leading the matcher profiling and plan estimation now underway (T13, T14). Both members co-authored the D.2.1, D.4.1 and D.4.2 documents and this report.

3. Demonstration of Completed Work

The live walkthrough follows the order of the completed tasks, so that each claim in Section 1 is shown working in the system. Each demonstration item is labelled with the task it verifies and the artifact the evaluator will see, as shown in the table.

Demo	Verifies	What is shown live
D1	T1	`docker compose up` starts mongod, mongot and Ollama; the app connects and lists the running models
D2	T2	Source collections in MongoDB with original attributes and derived fields (title_norm, tokens, bkey, embedding) on a sample record from each dataset
D3	T3–T6	The four index definitions, then one blocking pipeline per path producing candidate pairs for the same request
D4	T7	explain("executionStats") for each path, showing the index actually used, and the profiling entries in run_log
D5	T8–T10	The three classical matchers scoring the same candidate pairs, with pairs above, below and between the thresholds
D6	T11	An uncertain pair escalated to the local LLM; the same pair re-submitted is answered from llm_cache with no model call
D7	T12	The Streamlit prototype running a full request under a fixed plan, from dataset choice to entity clusters

The demonstration focuses on the database behaviour that the project is built around. Item D4 shows that blocking runs as index-backed aggregation pipelines inside MongoDB rather than as application code, which is the basis of treating blocking as access-path selection. Item D6 shows the verdict cache as a materialized collection: repeating a request produces no new LLM calls, which is the property required by NFR-3. Item D7 shows that one logical request already executes under interchangeable physical operators, since the same request can be run with any access path and any subset of matchers. This is logical–physical data independence in practice, and it is exactly the decision the optimizer will take over in Phase 3.

Next steps. By November 15 the team will complete matcher profiling and plan estimation (T13, T14) and deliver plan enumeration with Pareto pruning (T15), so that the prototype runs the plan the optimizer selects and logs predicted against observed cost and F1 (T16). Transitivity repair and the full baseline evaluation (T17, T19) follow in Phase 4, ending November 29.

References

[1] S. Patel and N. Syed, “PlanER: Cost-Based Optimization of Hybrid Classical–LLM Entity Resolution Plans in a Document Database,” D.2.1 Full Project Proposal, COMP 8157, University of Windsor, Oct. 2026.
[2] S. Patel and N. Syed, “D.4.1 User Requirements and Analysis Document” and “D.4.2 Design Document,” COMP 8157, University of Windsor, Oct. 2026.
[3] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” J. Amer. Statist. Assoc., vol. 64, no. 328, pp. 1183–1210, 1969.
[4] L. Patel et al., “Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS,” Proc. VLDB Endow., vol. 18, no. 11, pp. 4171–4184, 2025.
[5] P. G. Selinger et al., “Access path selection in a relational database management system,” in Proc. ACM SIGMOD, 1979, pp. 23–34.
[6] MongoDB, Inc., “Public preview: MongoDB Community Edition now offers native full-text and vector search,” Sep. 2025.